

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №1
по дисциплине «Математическая логика и теория
формальных языков»
«Лексический анализатор»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Востров А. В. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Структура транслятора	4
1.2 Лексический анализ	4
1.2.1 Назначение лексического анализа	4
1.2.2 Классификация символов	4
1.3 Формальная модель лексического анализатора	5
1.4 Синтаксические диаграммы	6
1.5 Обработка ошибок	6
2 Особенности реализации	8
2.1 Структуры данных	8
2.2 Методы	8
2.2.1 Метод tokenize	8
2.2.2 Метод print_table	8
2.2.3 Метод print_errors	8
2.3 Использование регулярных выражений	9
3 Результаты работы программы	10
Заключение	12
Список литературы	13
Приложение А. Код программы	14

Введение

Лексический анализ – это первый этап обработки текста программы, на котором исходный текст программы преобразуется в последовательность лексем. Лексический анализатор читает входной поток символов исходного кода, группирует их в лексемы, и определяет тип каждой лексемы согласно правилам лексической грамматики входного языка. Лексемы могут быть ключевыми словами, идентификаторами, константами, операторами, разделителями.

В данной лабораторной работе необходимо разработать программу, которая выполняет лексический анализ входного текста в соответствии с заданными требованиями. Программа должна:

- Разбивать входной текст на лексемы,
- Классифицировать лексемы по типам,
- Порождать таблицу лексем с указанием их типов и значений,
- Выдавать сообщения о наличии во входном тексте ошибок, которые могут быть обнаружены на этапе лексического анализа,
- Проверять длину идентификаторов и строковых констант на соответствие ограничениям (не более 15 символов),
- Обрабатывать комментарии произвольной длины.

Вариант 8: Входной язык содержит только кириллицу, арифметические выражения, разделенные символом ; (точка с запятой). Арифметические выражения состоят из идентификаторов, символьных констант (один символ в одинарных кавычках), строковых констант (символы в двойных кавычках), знака присваивания ($:=$), знаков операций $+$, $-$, $*$, $/$ и квадратных скобок.

1 Математическое описание

1.1 Структура транслятора

Транслятор осуществляет преобразование входного текста в выходные данные. Транслятор строится добавлением семантических действий, которые выполняются по мере распознавания входной строки.

Процесс трансляции делится на два этапа:

- **Распознавание** – Строится структура входной цепочки на основе порождающей грамматики входного языка. На этом этапе входные данные преобразуются в промежуточную структуру.
- **Генерация** – Промежуточная структура преобразуется в выходные данные.

1.2 Лексический анализ

Лексический анализ – это первая фаза трансляции, которая представляет собой предварительную обработку входного текста с выделением в нем лексем – минимальных единиц языка, имеющих смысл.

- В естественном языке лексемами являются слова (словоформы).
- В языках программирования – идентификаторы, ключевые слова, константы, составные операторы.

1.2.1 Назначение лексического анализа

Лексический анализатор определяет:

- **Значение лексемы**, определяющее подстроку символов входной цепочки, соответствующих распознанному классу лексемы;
- **Класс лексемы**, определяющий общее название для категории элементов, обладающих общими свойствами.

1.2.2 Классификация символов

Лексический анализатор обрабатывает входную цепочку, а на его вход подаются символы, сгруппированные по категориям. Поэтому перед лексическим анализом осуществляется дополнительная обработка, сопоставляющая с каждым символом его класс.

Транслитератор – устройство, осуществляющее сопоставление класса с каждым отдельным символом.

Классы символов:

- буква – символы, означающие буквы;
- цифра – символы 0-9;
- разделитель – пробел, перевод строки, возврат каретки;
- игнорируемый – символы, встречающиеся во входном потоке, но игнорируемые;
- запрещенный – символы, которые не относятся к алфавиту языка, но встречаются во входной цепочке;
- прочие – символы, не вошедшие ни в одну из определенных категорий.

1.3 Формальная модель лексического анализатора

Лексический анализатор – это конечный автомат, который преобразует входную строку символов в последовательность токенов.

Лексический анализатор реализует отображение:

$$F_{lexor} : \Sigma^* \rightarrow (T \times D)^*$$

где:

- Σ – входной алфавит (множество допустимых символов),
- T – множество типов токенов,
- D – множество значений токенов.

Тогда:

- Σ^* – множество всех возможных строк над алфавитом Σ ,
- $(T \times D)^*$ – множество последовательностей пар (тип токена, значение).

В данной работе:

- $\Sigma = \Sigma_{\text{кир}} \cup \Sigma_{\text{спец}}$, где
 - $\Sigma_{\text{кир}} = \{A, \dots, Я, a, \dots, я\}$;
 - $\Sigma_{\text{спец}} = \{+, -, *, /, [,], :, =, ;, ', ", \#, \backslash t, \backslash n, \backslash r\}$.
- $T = \{\text{идентификатор, знак присваивания, оператор, } [,], \text{разделитель, символьная константа, строковая константа}\}$.

- $D = D_{id} \cup D_{char} \cup D_{str} \cup D_{op} \cup D_{sep}$, где
 - $D_{id} = \{w \in \Sigma_{\text{кир}}^+ : |w| \leq 15\}$,
 - $D_{char} = \{w \in \Sigma \setminus \{', \backslash t, \backslash n, \backslash r\} : |w| = 1\}$
 - $D_{str} = \{w \in (\Sigma \setminus \{', \backslash t, \backslash n, \backslash r\})^* : |w| \leq 15\}$
 - $D_{op} = \{+, -, *, /, :=\}$;
 - $D_{sep} = \{[,], ;\}$;

1.4 Синтаксические диаграммы

Синтаксические диаграммы – формы представления правил формирования лексем. Синтаксическая диаграмма работы лексического анализатора, рассматривающегося в данной работе, представлена на [Рис. 1](#).

```

y1: Выдать лексему "+"
y2: Выдать лексему "-"
y3: Выдать лексему "*"
y4: Выдать лексему "/"
y5: Выдать лексему "["
y6: Выдать лексему "]"
y7: Выдать лексему ":="
y8: Выдать лексему ";"
y9: Записываем символьную константу, выдаем лексему
<char_const, значение>
y10: Записываем строковую константу, выдаем лексему
<string_const, значение>
y11: Символ в буфер
y12: Записываем идентификатор, выдаем лексему <id, имя>
WS (Whitespace): Пропуск пробельных символов (пробел, табуляция,
перевод строки).

```

Лексемами являются идентификаторы, знак присваивания, операторы $+$, $-$, $*$, $/$, левая квадратная скобка $[$, правая квадратная скобка $]$, символьные константы, строковые константы, разделитель $;$.

1.5 Обработка ошибок

Лексический анализатор обнаруживает следующие ошибки:

- Недопустимые символы;
- Длина идентификаторов и строковых констант более 15 символов;

- Незакрывающая строковая константа и незакрывающая символьная константа;
- Длина символьных констант не более 1 символа.

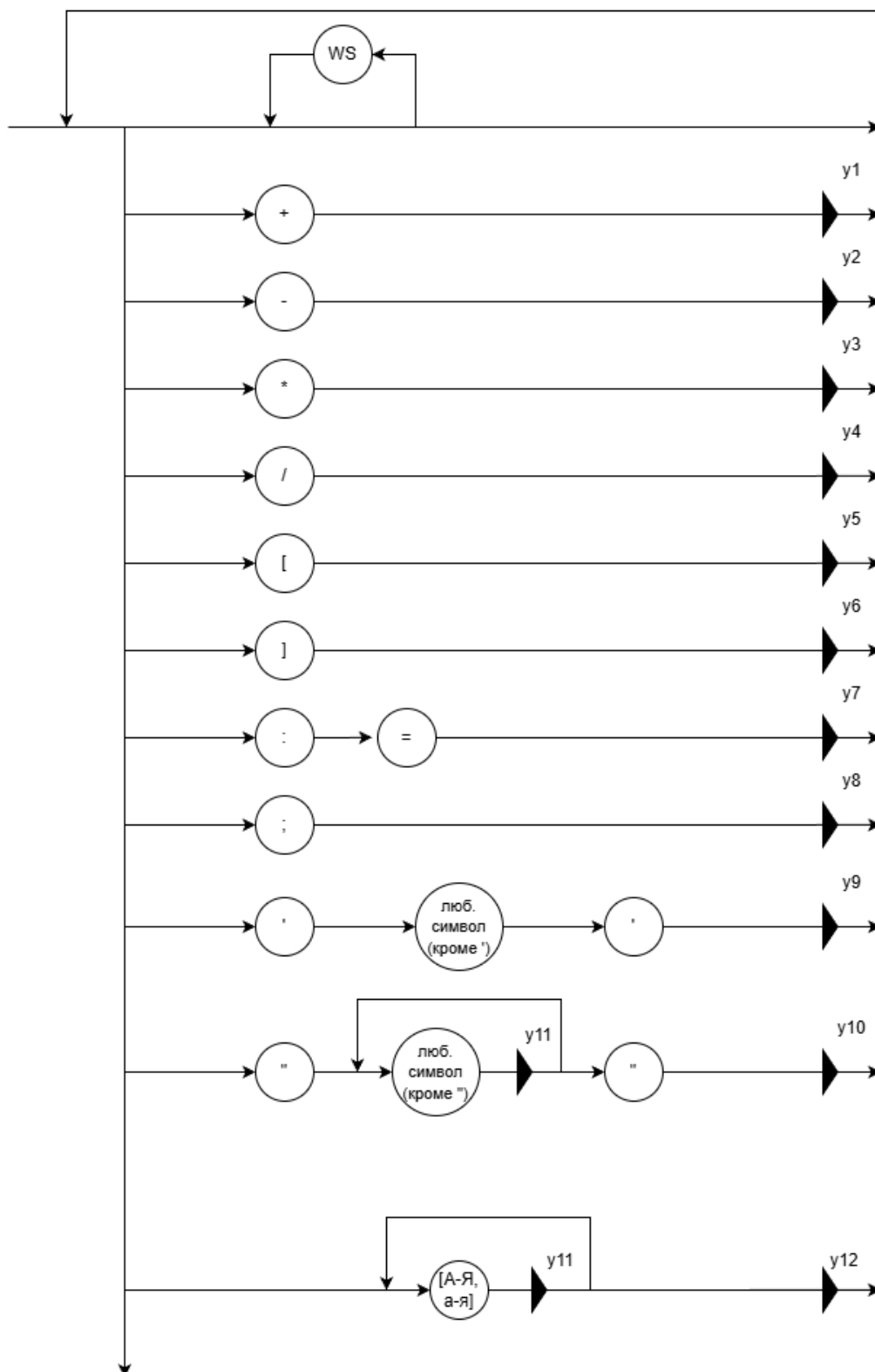


Рис. 1. Синтаксическая диаграмма

2 Особенности реализации

2.1 Структуры данных

- `text` – входная цепочка символов;
- `tokens` – список токенов (кортежей в формате (лексема, тип, значение));
- `symbol_table` – таблица символов, в которой хранятся уникальные идентификаторы в формате {имя: порядковый номер};
- `errors` – список ошибок;
- `token_specification` – список правил распознавания, связывающий типы лексем с regex-паттернами.

2.2 Методы

2.2.1 Метод `tokenize`

Вход: `text` – входная цепочка символов.

Выход: `tokens` – последовательность кортежей (лексема, тип, значение).

Метод последовательно обрабатывает входную строку, сопоставляя символы с паттернами из списка. Для каждого найденного совпадения он определяет тип лексемы: пробельные символы игнорируются, некорректные фрагменты (незакрытые кавычки, недопустимые символы, слишком длинные идентификаторы или константы) фиксируются как ошибки. Если ошибок нет, метод записывает информацию о лексеме (тип, значение, номер в таблице символов) в общий список результатов, формируя итоговую последовательность токенов для дальнейшей обработки.

2.2.2 Метод `print_table`

Вход: `tokens` – последовательность кортежей (лексема, тип, значение).

Выход: напечатанная в консоли таблица лексем.

Метод выводит таблицу лексем с колонками: Лексема, Тип лексемы, Значение. Для идентификаторов показывает имя и порядковый номер из таблицы символов.

2.2.3 Метод `print_errors`

Вход: `errors` – список обнаруженных ошибок.

Выход: напечатанный в консоли список ошибок.

Метод выводит сообщения об ошибках, обнаруженных при трансляции, таких как незакрытые строковые/символьные константы, превышение длины идентификатора/строки, нераспознанные символы.

2.3 Использование регулярных выражений

Регулярные выражения в программе используются для распознавания лексем во входном тексте. Они отвечают за синтаксическую правильность лексем, определяя принадлежность входной подстроки языку. Формы лексем описываются правилами токенизации, которые объединяются в одно составное регулярное выражение.

Листинг 1. Генератор регулярного выражения

```
1 self.regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in self.  
    token_specification)
```

Правила токенизации описаны в списке кортежей `token_specification`.

Листинг 2. Правила токенизации

```
1 self.token_specification = [  
2     ('COMMENT',      r'#[^#]*#'),  
3     ('ASSIGN',       r':='),  
4     ('SEPARATOR',    r';'),  
5     ('L_BRACKET',    r'\['),  
6     ('R_BRACKET',    r'\]'),  
7     ('PLUS',         r'\+'),  
8     ('MINUS',        r'-'),  
9     ('MUL',          r'\*'),  
10    ('DIV',          r'/'),  
11    ('STRING_CONST',  r'"[A-Za-яЁё+\-*/\[\]:=;\s]*"'),  
12    ('STRING_OPEN',   r'"[A-Za-яЁё+\-*/\[\]:=;\s]*'),  
13    ('CHAR_CONST',    r"'[A-Za-яЁё+\-*/\[\]:=;\s]*'"),  
14    ('CHAR_OPEN',     r"'[A-Za-яЁё+\-*/\[\]:=;\s]*"),  
15    ('IDENTIFIER',    r'[A-Za-яЁё][A-Za-яЁё]*'),  
16    ('SKIP',          r'[\t\n\r]+'),  
17    ('MISMATCH',      r'.'),  
18 ]
```

Код программы приведен в [Приложении А](#).

3 Результаты работы программы

Далее представлены результаты работы программы для различных входных данных.

Вход: `a1 := '1'; b2 := "2"; v3 := a1 + b2; g4 := [v3 * a1]`

```
1 Входные данные: a1 := '1'; b2 := "2"; v3 := a1 + b2; g4 := [v3 * a1]
2
3 Таблица лексем:
4 Лексема          Тип лексемы          Значение
5 -----
6 a1                Идентификатор          a1 : 1
7 :=                Знак присваивания      :=
8 '1'               Символьная константа  '1'
9 ;                Разделитель            ;
10 b2                Идентификатор          b2 : 2
11 :=                Знак присваивания      :=
12 "2"              Строковая константа    "2"
13 ;                Разделитель            ;
14 v3                Идентификатор          v3 : 3
15 :=                Знак присваивания      :=
16 a1                Идентификатор          a1 : 1
17 +                Операция                +
18 b2                Идентификатор          b2 : 2
19 ;                Разделитель            ;
20 g4                Идентификатор          g4 : 4
21 :=                Знак присваивания      :=
22 [                Левая кв. скобка        [
23 v3                Идентификатор          v3 : 3
24 *                Операция                *
25 a1                Идентификатор          a1 : 1
26 ]                Правая кв. скобка        ]
```

Вход: `оченьдлинныйидентификатор := 'A'; короткая := "оченьдлиннаястрока"`

```
1 Входные данные: оченьдлинныйидентификатор := 'A'; короткая := "оченьдлинная
  строка"
2
3 Ошибки:
4 -----
5 Ошибка в позиции 0: длина идентификатора 'оченьдлинныйидентификатор' превыша
  ет 15 символов (25 симв.)
6 Ошибка в позиции 46: длина строковой константы '"оченьдлиннаястрока"' превыш
  ает 15 символов (20 симв.)
7
8 Таблица лексем:
9 Лексема          Тип лексемы          Значение
10 -----
11 :=                Знак присваивания      :=
12 'A'               Символьная константа  'A'
13 ;                Разделитель            ;
14 короткая          Идентификатор          короткая : 1
15 :=                Знак присваивания      :=
```

Вход: а := 'А

1 Входные данные: а := 'А

2

3 Ошибки:

4 -----

5 Ошибка в позиции 5: незакрытая символьная константа ''А

6

7 Таблица лексем:

Лексема	Тип лексемы	Значение
---------	-------------	----------

9 -----

а	Идентификатор	а : 1
---	---------------	-------

:=	Знак присваивания	:=
----	-------------------	----

Вход: # это комментарий # а1 ::=]] 'АБ'\ на := б @ в # г \$ д;;

1 Входные данные: # это комментарий # а1 ::=]] 'АБ'

2 а := б @ в # г \$ д;;

3

4 Ошибки:

5 -----

6 Ошибка в позиции 21: неожиданный символ '1'

7 Ошибка в позиции 23: неожиданный символ ':'

8 Ошибка в позиции 26: неожиданный символ '='

9 Ошибка в позиции 30: символьная константа должна содержать ровно 1 символ, а не 2 ('АБ')

10 Ошибка в позиции 42: неожиданный символ '@'

11 Ошибка в позиции 46: неожиданный символ '#'

12 Ошибка в позиции 50: неожиданный символ '\$'

13

14 Таблица лексем:

Лексема	Тип лексемы	Значение
---------	-------------	----------

16 -----

а	Идентификатор	а : 1
---	---------------	-------

:=	Знак присваивания	:=
----	-------------------	----

]	Правая кв. скобка	]
---	-------------------	---

]	Правая кв. скобка	]
---	-------------------	---

а	Идентификатор	а : 1
---	---------------	-------

:=	Знак присваивания	:=
----	-------------------	----

б	Идентификатор	б : 2
---	---------------	-------

в	Идентификатор	в : 3
---	---------------	-------

г	Идентификатор	г : 4
---	---------------	-------

д	Идентификатор	д : 5
---	---------------	-------

;	Разделитель	;
---	-------------	---

;	Разделитель	;
---	-------------	---

Заключение

В ходе выполнения данной лабораторной работы был реализован лексический анализатор для языка, состоящего из кириллицы и арифметических выражений, разделенных символом ; содержащих идентификаторы, символьные константы, строковые константы, знак присваивания ($:=$), знаки операций $+$, $-$, $*$, $/$ и $[]$. Также была составлена синтаксическая диаграмма к этому языку.

Достоинства:

- Легко добавить новый тип токена, так как в коде спецификация лексем представлена в виде списка кортежей и нет необходимости переделывать регулярные выражения.
- Вывод номера символа при ошибке.

Недостатки:

- Зависимость от кодировки, так как при работе с кириллицей в разных кодировках могут возникнуть ошибки с распознаванием символов диапазона [А-Яа-яЁё].
- При большом количестве ошибок вывод на экран занимает много времени.

Масштабирование:

- Добавление новых типов токенов, например, логических операций.

Список литературы

1. Секция «Телематика» – <https://tema.spbstu.ru/compiler/> (дата обращения – 04.03.2026)

Приложение А. Код программы

```
1 import re
2
3
4 class Lexer:
5     def __init__(self, text):
6         self.text = text
7         self.tokens = []
8         self.symbol_table = {}
9         self.id_counter = 1
10        self.errors = []
11
12        self.token_specification = [
13            ('COMMENT', r'#[^#]*#'),
14            ('ASSIGN', r':='),
15            ('SEPARATOR', r';'),
16            ('L_BRACKET', r'\['),
17            ('R_BRACKET', r'\]'),
18            ('PLUS', r'\+'),
19            ('MINUS', r'-'),
20            ('MUL', r'\*'),
21            ('DIV', r'/'),
22            ('STRING_CONST', r'"[A-Za-яЁё+\-*/\[\]]:=;#\s]*"'),
23            ('STRING_OPEN', r'"[A-Za-яЁё+\-*/\[\]]:=;#\s]*'),
24            ('CHAR_CONST', r"'[A-Za-яЁё+\-*/\[\]]:=;#\s]*'"),
25            ('CHAR_OPEN', r"'[A-Za-яЁё+\-*/\[\]]:=;#\s]*"),
26            ('IDENTIFIER', r'[A-Za-яЁё][A-Za-яЁё]*'),
27            ('SKIP', r'\t\n\r+'),
28            ('MISMATCH', r'.'),
29        ]
30
31        self.regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in
32                               self.token_specification)
33
34    def tokenize(self):
35        for mo in re.finditer(self.regex, self.text):
36            kind = mo.lastgroup
37            value = mo.group()
38            position = mo.start()
39
40            if kind == 'SKIP' or kind == 'COMMENT':
41                continue
42
43            elif kind == 'MISMATCH':
44                self.errors.append(f"Ошибка в позиции {position}: неожиданный символ '{value}'")
45                continue
46
47            lexeme_type = ""
48            output_value = value
49            is_error = False
50
51            # проверка на незакрытую строковую константу
52            if kind == 'STRING_OPEN':
53                self.errors.append(f"Ошибка в позиции {position}: незакрытая строковая константа \"{value}\"")
54                is_error = True
55
56            # проверка на незакрытую символьную константу
```

```

56         elif kind == 'CHAR_OPEN':
57             self.errors.append(f"Ошибка в позиции {position}: незакрытая
                               символная константа '{value}")
58             is_error = True
59
60         # проверка длины идентификатора
61         elif kind == 'IDENTIFIER':
62             if len(value) > 15:
63                 self.errors.append(
64                     f"Ошибка в позиции {position}: длина идентификатора
                       '{value}' превышает 15 символов ({len(value)} сим
                       в.)")
65                 is_error = True
66             else:
67                 lexeme_type = "Идентификатор"
68                 if value not in self.symbol_table:
69                     self.symbol_table[value] = self.id_counter
70                     self.id_counter += 1
71                 output_value = f"{value} : {self.symbol_table[value]}"
72
73         # проверка длины строковой константы
74         elif kind == 'STRING_CONST':
75             if len(value) > 15:
76                 self.errors.append(
77                     f"Ошибка в позиции {position}: длина строковой конст
                       анты '{value}' превышает 15 символов ({len(value)
                       } симв.)")
78                 is_error = True
79             else:
80                 lexeme_type = "Строковая константа"
81                 output_value = value[1:-1]
82
83         # проверка длины символьной константы
84         elif kind == 'CHAR_CONST':
85             content = value[1:-1]
86             if len(content) != 1:
87                 self.errors.append(
88                     f"Ошибка в позиции {position}: символная константа
                       должна содержать ровно 1 символ, а не {len(
                       content)} ('{content}')"
89                 is_error = True
90             else:
91                 lexeme_type = "Символьная константа"
92                 output_value = content
93
94         elif kind == 'ASSIGN':
95             lexeme_type = "Знак присваивания"
96         elif kind == 'SEPARATOR':
97             lexeme_type = "Разделитель"
98         elif kind == 'L_BRACKET':
99             lexeme_type = "Левая кв. скобка"
100        elif kind == 'R_BRACKET':
101            lexeme_type = "Правая кв. скобка"
102        elif kind in ['PLUS', 'MINUS', 'MUL', 'DIV']:
103            lexeme_type = "Оператор"
104
105        if not is_error and lexeme_type:
106            self.tokens.append((value, lexeme_type, output_value))
107
108    return self.tokens

```

```

109
110     def print_table(self):
111         print("\nТаблица лексем:")
112         print(f"{'Лексема':<15} {'Тип лексемы':<25} {'Значение'}")
113         print("-" * 65)
114         for lexeme, l_type, val in self.tokens:
115             print(f"{lexeme:<15} {l_type:<25} {val}")
116
117     def print_errors(self):
118         if self.errors:
119             print("\nОшибки:")
120             print("-" * 65)
121             for error in self.errors:
122                 print(error)
123
124
125 data = "a1 := '1'; 62 := \"2\"; в3 := a1 + 62; r4 := [в3 * a1]"
126 print(f"Входные данные: {data}")
127 lexer = Lexer(data)
128 lexer.tokenize()
129 lexer.print_errors()
130 lexer.print_table()

```